ASP.NET Core Security

Cross-Origin Resource Sharing (CORS)

1. The Same-Origin Policy (The Problem)

By default, web browsers enforce the **Same-Origin Policy (SOP)**. If a user opens a website at `https://mywebsite.com`, the browser allows the JavaScript on that page to make API calls back to `https://mywebsite.com`.

However, if the JavaScript tries to silently make an API call to a different domain (e.g., `https://api.otherdomain.com`), the browser blocks it. This prevents malicious sites from hijacking a user's browser to execute unauthorized actions on other sites where the user might be logged in.

Note: An "Origin" is the combination of the Protocol + Domain + Port (e.g., `https://api.com:443`).

2. What is CORS? (The Solution)

CORS (Cross-Origin Resource Sharing) is a mechanism that allows the server to safely bypass the Same-Origin Policy. It provides a way for an API to instruct the browser that specific external origins are trusted.

> **CRITICAL DISTINCTION**
>
> CORS protects the **browser**, not the server. Tools like Postman or cURL will never encounter a CORS error because they are not web browsers enforcing the Same-Origin Policy. CORS only applies when JavaScript running inside a browser attempts a cross-origin request.

3. How CORS Works (The Preflight)

CORS operates via HTTP Headers and categorizes requests into two types:

- **Simple Requests (GET, POST with standard headers):** The browser sends the request with an `Origin` header. The server responds with the data and an `Access-Control-Allow-Origin` header. If the origins match, the browser hands the data to the JavaScript.
- **Complex Requests (PUT, DELETE, custom Authorization headers):** Before sending the actual request, the browser sends an invisible `OPTIONS` request called the **Preflight**. It asks the server for permission to execute a specific method or send a specific header. If the server grants permission, the browser proceeds with the actual request.

4. Configuration Options

When defining a CORS policy in ASP.NET Core, you control four primary parameters:

- **Origins:** Which domains are allowed? (e.g., `.WithOrigins("https://app.com")`)
- **Methods:** Which HTTP verbs are permitted? (e.g., `.WithMethods("GET", "POST")`)
- **Headers:** Which custom headers (like `Authorization`) can the client send?
- **Credentials:** Are cookies allowed cross-origin? (`.AllowCredentials()`). *Note: If credentials are allowed, you cannot use* `AllowAnyOrigin()`*; you must specify exact origins for security reasons.*

5. Implementation in ASP.NET Core

```
/*
 * ========================================================================
 * ASP.NET CORE SECURITY: CORS CONFIGURATION
 * ========================================================================
 * PIPELINE PLACEMENT RULE:
 * `app.UseCors()` MUST be placed after `app.UseRouting()` but before
 * `app.UseAuthorization()`. The CORS middleware needs to know which
 * route is being executed, but it must reply to the OPTIONS preflight
 * request before the Authorization middleware demands a token!
 * ========================================================================
 */
var builder = WebApplication.CreateBuilder(args);

// 1. REGISTER THE CORS POLICIES
builder.Services.AddCors(options =>
{
    // A strict policy for production applications
    options.AddPolicy("FrontendAppOnly", policy =>
    {
        policy.WithOrigins("https://my-react-app.com", "http://localhost:3000")
              .WithMethods("GET", "POST", "PUT", "DELETE")
              .WithHeaders("Authorization", "Content-Type")
              .AllowCredentials(); // Allows cross-origin cookies
    });
});

var app = builder.Build();

app.UseRouting();

// 2. APPLY THE POLICY
app.UseCors("FrontendAppOnly");

app.UseAuthentication();
app.UseAuthorization();

// 3. MAP ENDPOINTS
app.MapGet("/api/data", () => "Data fetched successfully");

app.Run();
```